

Reviewer Pack — Gromov 4-Point Hyperbolicity of Gate-Graph Caps $\text{ACC}^0[2]$ Bias...

Entry dd0beadbf6d6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 21:12:40 UTC

Gromov 4-Point Hyperbolicity of Gate-Graph Caps $\text{ACC}^0[2]$ Bias on MOD_3

Entry ID: dd0beadbf6d6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 21:12:40 UTC

1. Conjecture

Field A (mathematical branch): Coarse / Gromov hyperbolic geometry — the 4-point Gromov hyperbolicity $\delta(G) := (1/2) \cdot \sup_{\{a,b,c,d \in V(G)\}} [d(a,c)+d(b,d) - \max(d(a,b)+d(c,d), d(a,d)+d(b,c))]$ of a finite undirected graph with hop distance (Gromov 1987 ‘Hyperbolic groups’; Bridson-Haefliger 1999 ‘Metric Spaces of Non-Positive Curvature’; Chepoi-Dragan-Estallon-Habib-Vaxes 2008 ‘Diameters, centers, and approximating trees of δ -hyperbolic geodesic spaces’). Computed in $O(|V|^4)$ by brute-force 4-tuple enumeration over the all-pairs BFS distance matrix; the functional uses max, abs, and integer shortest-path arithmetic with no polynomial F_q extension (shortest-path distance is not preserved under polynomial ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe. The invariant is STRUCTURAL on the circuit DAG (two $\text{ACC}^0[2]$ circuits computing the same Boolean function can have very different δ), shielding from Razborov-Rudich natural proofs and avoiding the truth-table Fourier trap that killed Plünnecke doubling on top-Fourier spectra. An arXiv search ‘Gromov hyperbolicity’ AND (‘ ACC^0 ’ OR ‘Williams algorithmic method’ OR ‘circuit lower bound’ OR ‘Boolean circuit’) returns 0 direct hits and <5 adjacent papers (network coarse geometry on AS-level Internet graphs; QC analysis on Cayley graphs — never on ACC circuits). Distinct from prior structural circuit attempts: Mostar (edge distance-asymmetry on single edges, not 4-point Gromov product), Forman-Ricci (Bochner edge-vertex local sum), Sinkhorn/Birkhoff (doubly-stochastic spectral scaling), Chang spectrum ($\mathbb{Z}/p$ Fourier on DFS labels), additive energy (out-degree L^2), Sidon defect (max representation in F_2^N), fatgraph genus (oriented surface embedding) — none uses the 4-point Gromov product on the gate-graph. **Field B** (complexity object): $\text{ACC}^0[m]$ circuit size and depth for MOD_q on n inputs with q coprime to m (focus $q=3, m=2$), in the Williams 2011 / Williams 2014 / Murray-Williams 2018 algorithmic-method regime targeting $\text{NEXP} \not\subseteq \text{ACC}^0$ and explicit-function $\text{ACC}^0[2]$ lower bounds. Bridged via the Razborov-Smolensky non-approximability of MOD_3 by low-degree F_2 polynomials: any $\text{ACC}^0[2]$ circuit with non-trivial bias toward MOD_3 must wire its gate-graph in a way that prevents global tree-like (low- δ) coarse geometry, giving a structural obstruction independent of the truth-table side.

Statement:

Let C be a connected $\text{ACC}^0[2]$ circuit of depth d and gate-count s on n inputs, let G_C be the underlying undirected gate-adjacency graph with hop distance, and let $\delta(C) := (1/2) \cdot \max_{\{a,b,c,e \in V(G_C)\}} [d(a,c)+d(b,e) - \max(d(a,b)+d(c,e), d(a,e)+d(b,c))]$. Define the MOD_3 bias $B(C) := 2 \cdot \Pr_{x \in \{0,1\}^n} [C(x) = \text{MOD}_3(x)] - 1$. Then for every $n \geq 6$, $|B(C)|^2 \cdot n \leq (1 + \delta(C)) \cdot 2^{\{d\}}$, with a single (C, n) violating this inequality refuting the conjecture.

Rationale (proposer’s reasoning):

Gromov hyperbolicity measures large-scale tree-likeness of finite graphs and is bounded by diameter/2; for constant-depth $\text{ACC}^0[2]$ circuits the gate-graph is shallow but its δ can still detect modular obstructions because computing MOD_3 (Razborov-Smolensky-hard for F_2 polynomials) forces the circuit to wire many quasi-isometric ‘modular cycles’ through MOD_2 gates, raising δ . The $1+\delta$ factor lets tree-like ($\delta=0$) circuits saturate the bound only when $2^d \geq n \cdot |B|^2$, matching the NC^1 regime where balanced binary trees of depth $\log n$ can compute MOD_3 exactly while remaining 0-hyperbolic. Because δ depends only on the DAG wiring (not the truth table), the invariant is shielded from natural proofs; because shortest-path distance has no polynomial F_q extension, it is shielded from algebrization.

Taxonomy category: $\text{ACC_LB_via_WILLIAMS}$ (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 12e810a0a0e0650d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times$ 81 parameter combos $(d,s,n) \in \{2,3,4\} \times \{12,18,24\} \times \{8,10,12\}$, 6 circuits each (14,580 total), compute $\text{LHS/RHS} = |B(C)|^2 \cdot n / ((1+\delta(C)) \cdot 2^d)$. Conjecture SUPPORTED iff max ratio over all circuits ≤ 1.0 ; FALSIFIED iff any single (seed, circuit) yields ratio > 1.0 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP LIPTON	SAFE	0.85	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Gromov hyperbolicity ACC circuit lower bound MOD_3 -4-point delta hyperbolic Boolean circuit gate graph Razborov Smolensky-coarse geometry circuit complexity Williams algorithmic method hyperbolicity

Top relevant hits considered: - [<http://arxiv.org/abs/2504.19966v3>] Quantum circuit lower bounds in the magic hierarchy - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/2109.03725v3>] Quasi-metric antipodal spaces and maximal Gromov hyperbolic spaces - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/1601.02745v1>] Basic Reasoning with Tensor Product Representations - [<http://arxiv.org/abs/0711.2360v2>] Every longest circuit of a 3-connected, $K_{3,3}$ -minor free graph has a chord - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/0707.1469v3>] Coarse and synthetic Weil-Petersson geometry: quasi-flats, geodesics, and relative hyperbolicity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
from collections import deque

def matrix_mult(a, b):
    return [[sum(a[i][k] * b[k][j] for k in range(len(b))) for j in range(len(b[0]))] for i in range(len(a))]

def matrix_pow(mat, power):
    result = [[0] * len(mat) for _ in range(len(mat))]
    for i in range(len(mat)):
        result[i][i] = 1
    while power > 0:
        if power % 2 == 1:
            result = matrix_mult(result, mat)
        mat = matrix_mult(mat, mat)
        power //= 2
    return result

def compute_delta(graph):
    n = len(graph)
    if n == 0:
        return 0.0

    # Compute all-pairs shortest paths using Floyd-Warshall
    dist = [[float('inf')] * n for _ in range(n)]
    for i in range(n):
        dist[i][i] = 0
        for j in graph[i]:
            dist[i][j] = 1

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    max_delta = 0.0
    for a in range(n):
        for b in range(n):
            for c in range(n):
                for e in range(n):
                    if a == b or a == c or a == e or b == c or b == e or c == e:
                        continue
                    d_ac = dist[a][c]
                    d_be = dist[b][e]
                    d_ab = dist[a][b]
                    d_ce = dist[c][e]
                    d_ae = dist[a][e]
```

```

        d_bc = dist[b][c]
        delta = 0.5 * (d_ac + d_be - max(d_ab + d_ce, d_ae + d_bc))
        if delta > max_delta:
            max_delta = delta

    return max_delta

def generate_circuit(d, s, n):
    layers = [[] for _ in range(d + 1)]
    for i in range(n):
        layers[0].append(('input', i))

    gate_types = ['AND', 'OR', 'NOT', 'MOD_2']
    for layer in range(1, d + 1):
        num_gates = min(s, random.randint(2, 4))
        for _ in range(num_gates):
            gate_type = random.choice(gate_types)
            fan_in = random.randint(2, min(4, len(layers[layer - 1])))
            inputs = random.sample(layers[layer - 1], fan_in)
            layers[layer].append((gate_type, inputs))

    output_layer = layers[-1]
    if not output_layer:
        return None, None

    output_gate = random.choice(output_layer)
    return layers, output_gate

def evaluate_circuit(circuit, x):
    if not circuit:
        return 0

    layers, output_gate = circuit
    values = [x]

    for layer in range(1, len(layers)):
        layer_values = []
        for gate in layers[layer]:
            gate_type, inputs = gate
            input_values = [values[prev_layer][i] for prev_layer, i in inputs]
            if gate_type == 'AND':
                val = all(input_values)
            elif gate_type == 'OR':
                val = any(input_values)
            elif gate_type == 'NOT':
                val = not input_values[0]
            elif gate_type == 'MOD_2':
                val = sum(input_values) % 2
            else:
                val = 0
            layer_values.append(val)
        values.append(layer_values)

    output_layer = len(layers) - 1
    output_index = layers[output_layer].index(output_gate)
    return values[output_layer][output_index]

```

```

def compute_bias(circuit, n):
    if not circuit:
        return 0.0

    total = 0
    count = 0
    for _ in range(2 ** n):
        x = [random.randint(0, 1) for _ in range(n)]
        mod3 = sum(x) % 3
        output = evaluate_circuit(circuit, x)
        if output == mod3:
            total += 1
            count += 1

    if count == 0:
        return 0.0
    return 2.0 * (total / count) - 1.0

def build_graph(circuit):
    if not circuit:
        return []

    layers, _ = circuit
    nodes = []
    node_indices = {}
    index = 0

    for layer in layers:
        for gate in layer:
            nodes.append(gate)
            node_indices[gate] = index
            index += 1

    graph = [[] for _ in range(len(nodes))]
    for i, gate in enumerate(nodes):
        if isinstance(gate[1], list):
            for input_gate in gate[1]:
                j = node_indices[input_gate]
                graph[i].append(j)
                graph[j].append(i)

    return graph

def run_trial(seed):
    random.seed(seed)
    d_values = [2, 3, 4]
    s_values = [12, 18, 24]
    n_values = [8, 10, 12]

    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for d in d_values:
        for s in s_values:

```

```

    for n in n_values:
        for _ in range(6):
            circuit = generate_circuit(d, s, n)
            if not circuit:
                continue

            graph = build_graph(circuit)
            delta = compute_delta(graph)
            bias = compute_bias(circuit, n)

            lhs = (bias ** 2) * n
            rhs = (1 + delta) * (2 ** d)

            if rhs == 0:
                continue

            ratio = lhs / rhs
            metric_values.append(ratio)
            instances_tested += 1

            if ratio > 1.0:
                conjecture_holds = False
                counterexample = f"d={d}, s={s}, n={n}, ratio={ratio:.4f}"

if not metric_values:
    return {
        "metric_name": "max_LHS/RHS_ratio",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

max_ratio = max(metric_values)
if max_ratio > 1.0:
    conjecture_holds = False

return {
    "metric_name": "max_LHS/RHS_ratio",
    "metric_value": max_ratio,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = sys.argv[1:]
    if not seeds:
        seeds = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79]

    metric_values = []
    instances_tested = 0
    conjecture_holds_all = True
    counterexample = ""

    for seed in seeds:

```

```

trial = run_trial(int(seed))
print(f"TRIAL: {trial}")
metric_values.append(trial["metric_value"])
instances_tested += trial["instances_tested"]
if not trial["conjecture_holds"]:
    conjecture_holds_all = False
    if not counterexample:
        counterexample = trial["counterexample"]

if not metric_values:
    print("RESULT: INCONCLUSIVE reason=no_metric_values")
    sys.exit(0)

mean = sum(metric_values) / len(metric_values)
variance = sum((x - mean) ** 2 for x in metric_values) / len(metric_values)
std = math.sqrt(variance)

support_fraction = sum(1 for x in metric_values if x <= 1.0) / len(metric_values)

if not conjecture_holds_all:
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[0]}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean:.4f} std={std:.4f} support_fraction={support_fraction}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4335b342.py", line 229, in <module>
    trial = run_trial(int(seed))
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4335b342.py", line 179, in run_trial
    graph = build_graph(circuit)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4335b342.py", line 147, in build_graph
    node_indices[gate] = index
    ~~~~~
TypeError: unhashable type: 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` ('unhashable type: list') in `build_graph` before any ratios were computed, so no data exists to evaluate the pre-registered support or falsification condition. | next: Fix `build_graph` to use a hashable node identifier (e.g., gate id or tuple) instead of a list, then rerun the full 14,580-circuit sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	550367
2	preregistration	claude_max	opus	0	0	7055
3	novelty	claude_max	opus	0	0	3777
4	novelty	claude_max	opus	0	0	8793
5	test_gen	mistral	codestral-latest	0	0	312860
6	test_gen	mistral	codestral-latest	0	0	317510
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	287736
8	test_gen	mistral	codestral-latest	0	0	316319
9	judge	claude_max	opus	0	0	4960

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1809376 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/dd0beadbf6d6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dd0beadbf6d6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dd0beadbf6d6.tar.gz` (if generated)